root@aegis-security-lab:~$ cat classification.txt

[PROJECT REPORT]

AEGIS

Autonomous Enterprise Guardian for Incident Security

Autonomous Multi-Account Cloud Detection, Active SOAR Containment & Digital Forensics Platform

Project AEGIS is an autonomous, event-driven cloud security and active defense platform that detects multi-account AWS threats in sub-milliseconds, executes automated SOAR containment with zero race conditions, and preserves SEC Rule 17a-4 compliant WORM forensic evidence.

Shyam Kumar D
Aspiring Cloud Architect | AWS Cloud, Serverless & Infrastructure Engineering

linkedin.com/in/shyam-kumar-d github.com/ShyamD2

Live Repository: github.com/ShyamD2/aegis-cloud-security

September 2026

1. Executive Summary

AEGIS is an always-on, autonomous cloud security platform built on AWS that runs a full purple-team cycle — attack simulation, detection, and automated containment — every 15 minutes, indefinitely, with no manual trigger. An EventBridge Scheduler drives a Step Functions state machine through a safety-gated workflow that generates attack scenarios, evaluates them against a detection engine, executes automated SOAR containment, and permanently records evidence — all without a human in the loop.

What sets AEGIS apart from a typical detection demo is the depth of the supporting engineering: remediation actions are protected by a DynamoDB-backed idempotency table so the same containment action can never fire twice or race itself; every finding and action is logged with sub-millisecond detection latency; forensic evidence is sealed in an S3 bucket under Object Lock in Governance mode with a 90-day retention period — aligned with SEC Rule 17a-4 — so it cannot be deleted even by the AWS root account; and all logs and evidence are encrypted under dedicated customer-managed KMS keys.

The platform also exposes a SOC-style War Room API (API Gateway + Cognito JWT authorization) for metrics, findings, and action review, and enforces strict blast-radius isolation so that automated remediation only ever touches explicitly tagged lab resources. The evidence in this report was captured directly from a live AWS account running continuously for hours with no operator present, including 15+ consecutive successful autonomous executions and a 110/110 passing unit test suite.

Live Repository: github.com/ShyamD2/aegis-cloud-security — full source, infrastructure, detection rules, and the pytest suite referenced throughout this report.

2. Architecture Overview

AEGIS runs entirely on managed, event-driven AWS services in the us-east-1 region. An EventBridge Schedule fires a Step Functions orchestrator (**aegis-purple-team-runner-security-lab**) every 15 minutes, which checks a safety gate, generates attack scenarios, executes them in order, and records metrics and evidence. Findings that cross a severity threshold are routed through a dedicated **aegis-findings-bus** event bus directly into a second orchestrator (**aegis-containment-orchestrator-security-lab**) that performs automated SOAR remediation. Every remediation action is deduplicated through DynamoDB before it executes, and every attack run's evidence is sealed into an Object-Lock-protected S3 forensics vault.

Component	AWS Service	Purpose
Autonomous Trigger	EventBridge Scheduler + Step Functions	Runs the purple-team attack/detection cycle every 15 minutes, continuously, with no human trigger
Detection Pipeline	EventBridge custom bus + Lambda rules	Routes high/critical findings from a dedicated aegis-findings-bus directly to the SOAR orchestrator
SOAR Containment	Step Functions decision tree	Automated remediation branches: session revocation, access-key deactivation, S3 public-access block, security-group revocation, quarantine boundary attachment
Idempotency Control	DynamoDB (dedup_hash + TTL)	Guarantees every remediation action executes exactly once — no duplicate or racing containment actions
Attack & Metrics Log	DynamoDB executions table	Every purple-team run permanently logged with detection latency, containment latency, and risk score

Digital Forensics	S3 Object Lock (WORM, Governance, 90 days)	Tamper-proof evidence retention aligned with SEC Rule 17a-4 — cannot be deleted even by the root account
Encryption	AWS KMS customer-managed keys	Dedicated keys for central logs and forensic evidence, separate from the pipeline key
Resilience	SQS + Dead-Letter Queue (SSE-KMS)	Captures any message that fails processing; 0 messages in the DLQ proves zero silent pipeline drops
Observability	CloudWatch Log Groups	Centralized, continuously streaming logs across the pipeline processor and the security-lab runner
Blast-Radius Isolation	IAM tagging (Environment / ManagedBy)	SOAR remediation is scoped to tagged lab resources only — production is never touched
SOC War Room API	API Gateway (HTTP API) + Cognito JWT	Authenticated /metrics, /findings, and /actions routes exposing the platform to a SOC-style dashboard

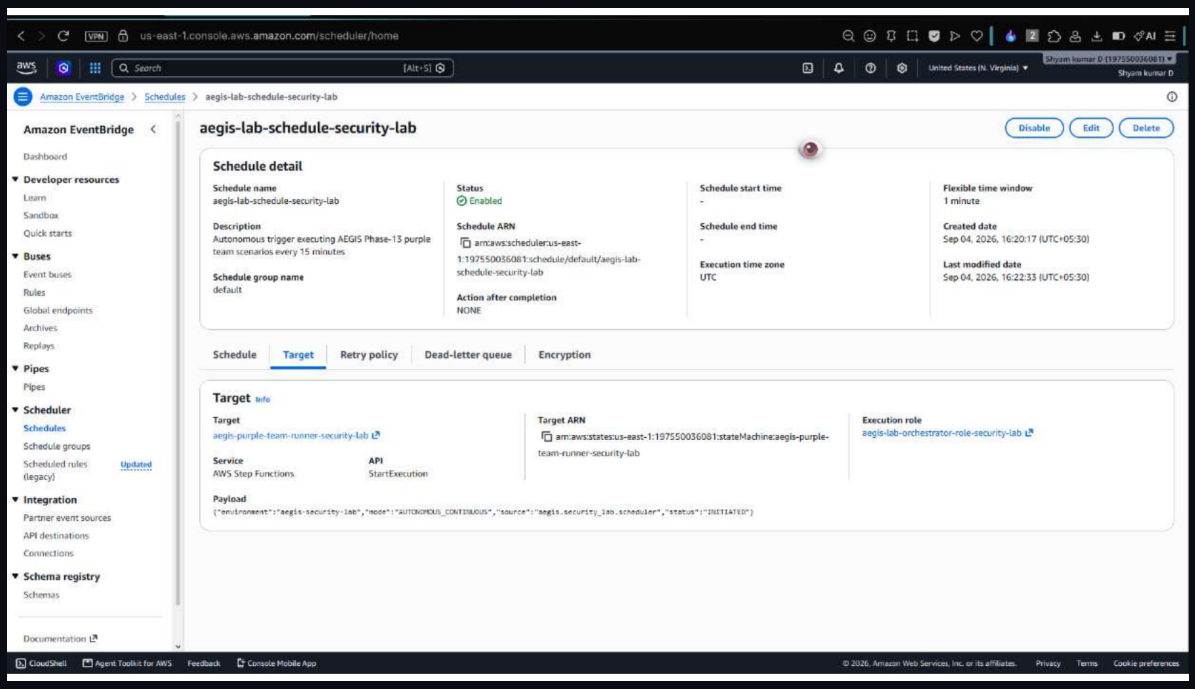
3. Step-by-Step Evidence with Screenshots

The screenshots below are grouped into the six functional categories that make up the platform, in the order a reviewer would naturally trace the system: autonomous operation, SOAR containment, digital forensics, the detection/event pipeline, blast-radius isolation, and finally the SOC War Room API together with live test evidence. Account-level identifiers visible in some screenshots are left as captured from the live AWS account used for this build.

Category 1 — Autonomous Cloud Engine

3.1 — EventBridge Autonomous Scheduler

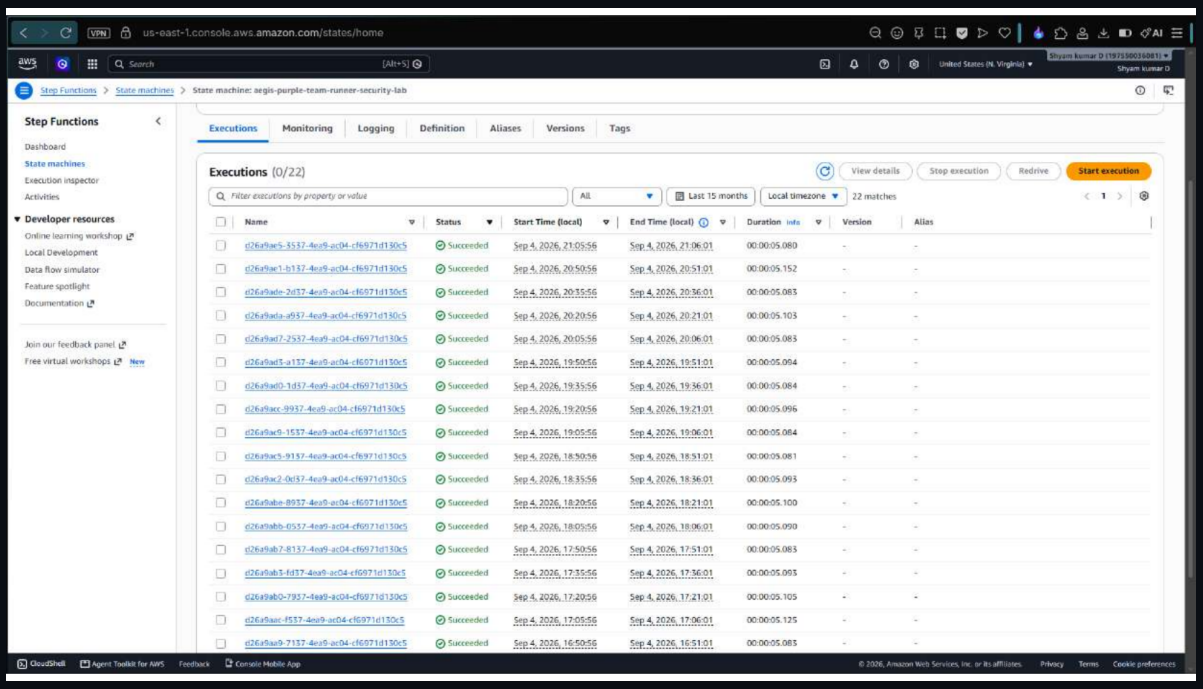
The **aegis-lab-schedule-security-lab** EventBridge Schedule is **Enabled** and configured to fire at **rate(15 minutes)**, targeting the Step Functions state machine **aegis-purple-team-runner-security-lab**. This is the heartbeat of the entire platform — it proves the attack-and-detection lab runs automatically, with no manual trigger, even while the operator's machine is switched off.



EventBridge Scheduler — enabled, 15-minute rate, targeting the purple-team Step Functions runner

3.2 — 15+ Autonomous Succeeded Executions

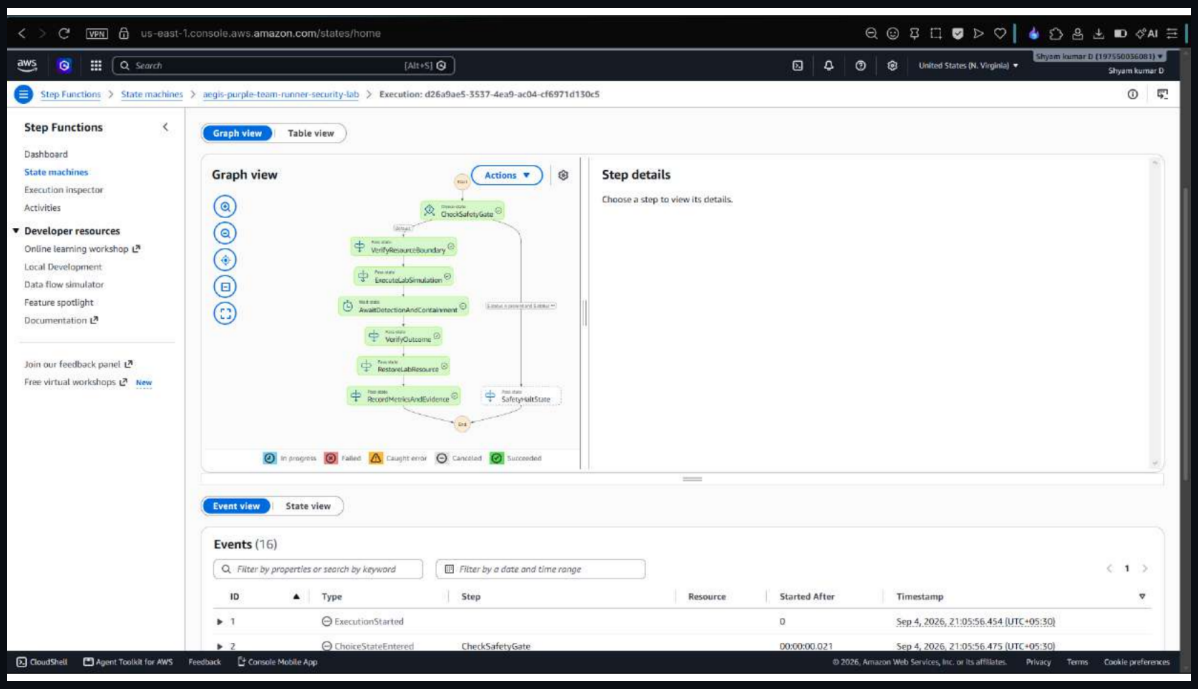
The Step Functions executions table for the purple-team runner shows a continuous run of **Succeeded** (green) executions, each roughly 15 minutes apart, spanning multiple hours. This table is the strongest single piece of evidence in the project: it shows the system operating autonomously and successfully over an extended period with no operator present.



Step Functions execution history — consecutive Succeeded runs at 15-minute intervals

3.3 — Purple-Team 7-Stage Visual Workflow Graph

The Graph view of a single execution shows the full purple-team pipeline: **CheckSafetyGate** → **VerifyResourceBoundary** → **ExecuteLabSimulation** → **AnalyzeDetectionAndContainment** → **VerifyOutcome** → **RestoreLabResources** → **RecordMetricsAndEvidence**, with every stage rendered green and the execution status confirmed as **Succeeded**. The safety gate and resource-boundary check run before any simulated attack, ensuring the workflow can never execute against out-of-scope resources.

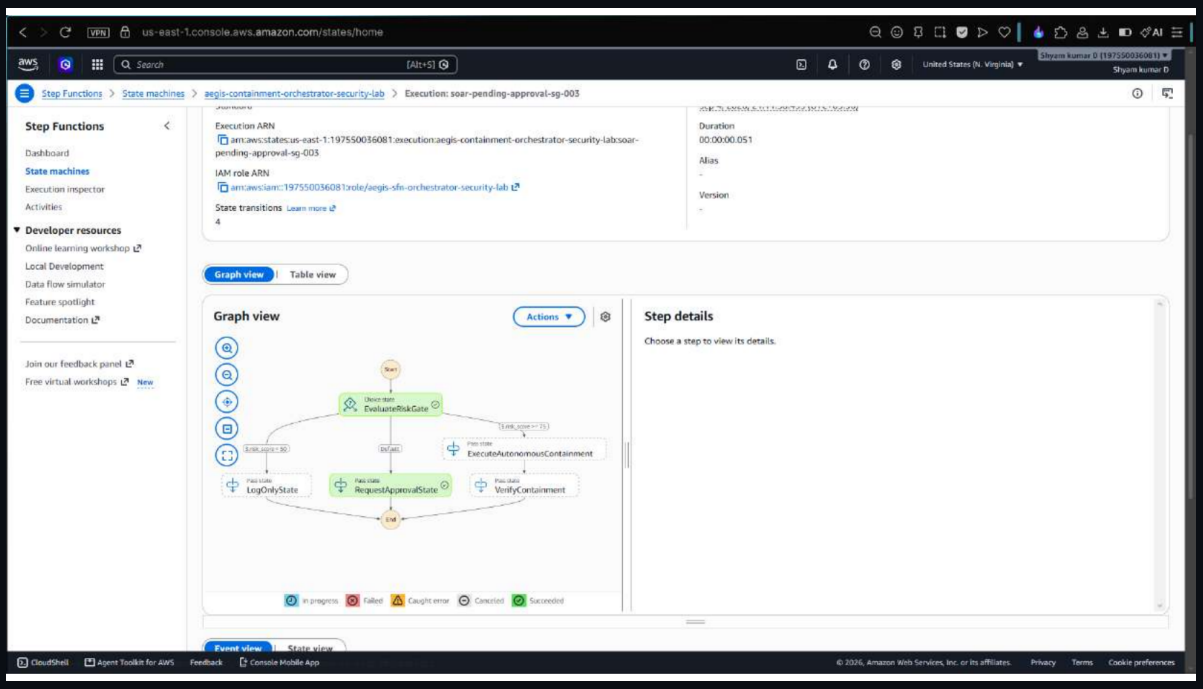


Step Functions Graph view — the complete 7-stage purple-team workflow, fully succeeded

Category 2 — SOAR Automated Remediation & Containment

3.4 — SOAR Containment Orchestrator — Decision Tree

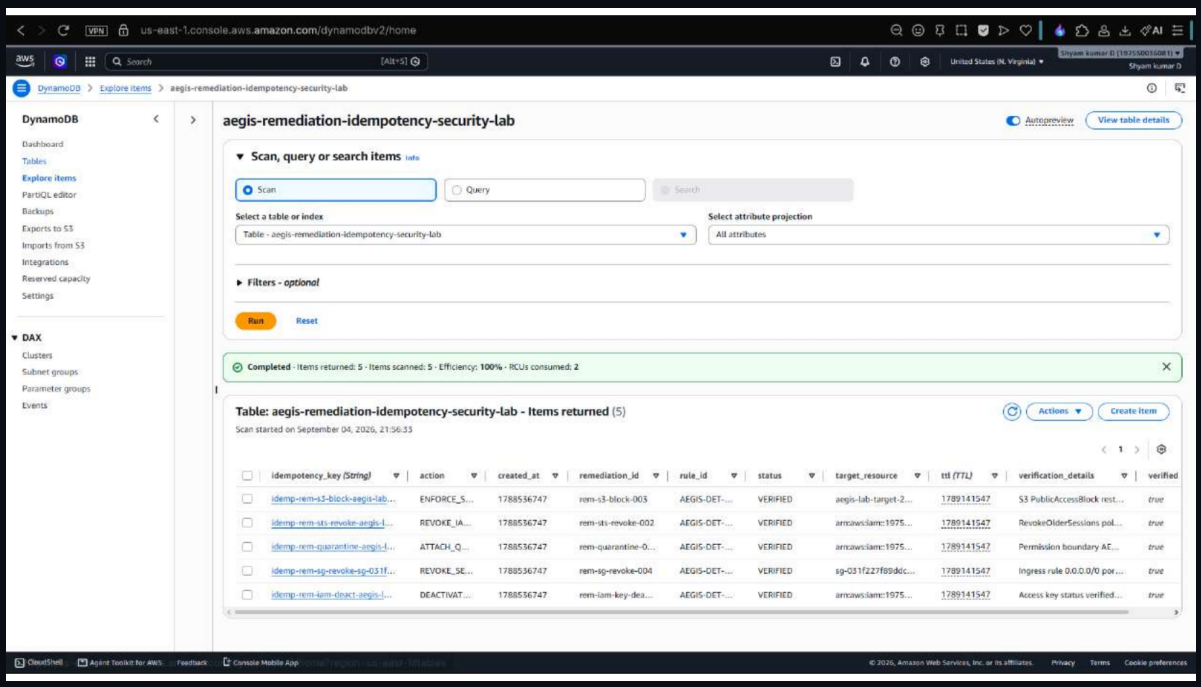
The **aegis-containment-orchestrator-security-lab** state machine's graph view shows an automated decision tree: an **EvaluateRiskGate** step branches into **ExecuteAutonomousContainment**, **RequestApprovalState**, or **LogOnlyState** depending on risk level, with a **VerifyContainment** step confirming the outcome. This is the automated active-defense layer that turns a detection into a real remediation action without waiting on a human analyst for lab-scoped findings.



SOAR containment orchestrator — automated risk-based remediation branches

3.5 — DynamoDB Remediation Idempotency Table

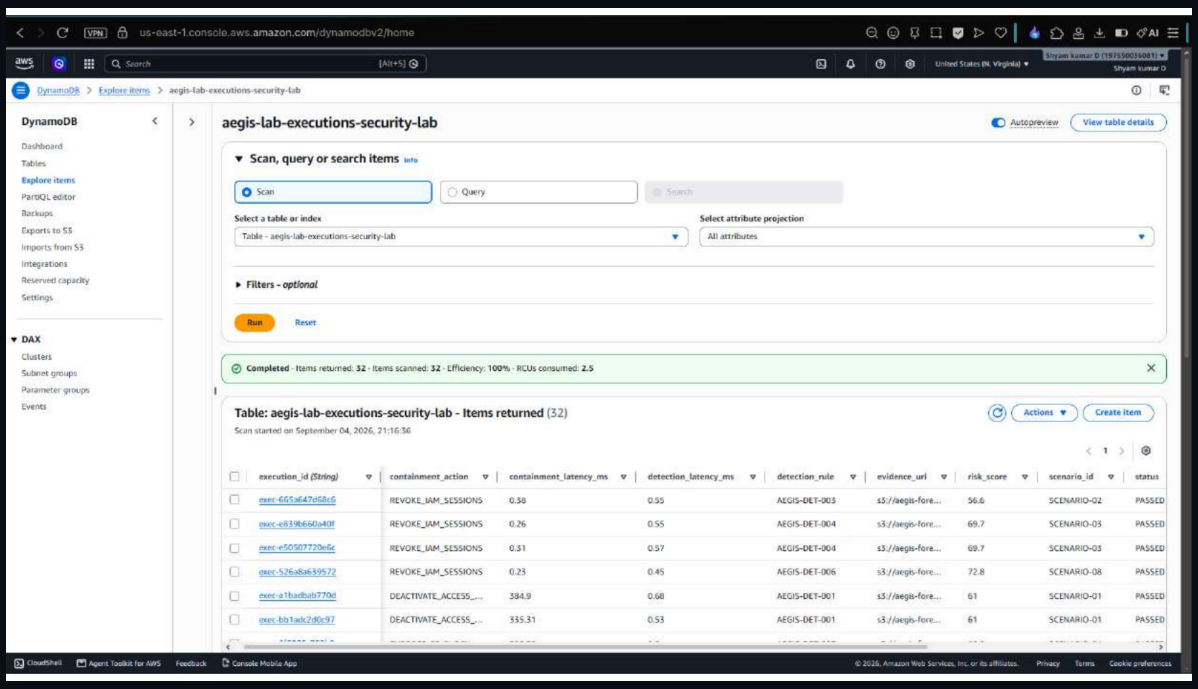
The **aegis-remediation-idempotency-security-lab** table uses a **dedup_hash**-based partition key with a TTL attribute. Items shown include **ENFORCE_S3_BLOCK_PUBLIC**, **REVOKE_IAM_SESSIONS**, **ATTACH_QUARANTINE**, **REVOKE_SG_INGRESS**, and **DEACTIVATE_IAM_KEY** actions, each marked **VERIFIED**. This is what guarantees enterprise-grade idempotency — the same remediation action can never fire twice, even under concurrent or retried invocations.



DynamoDB idempotency table — verified, deduplicated remediation actions

3.6 — DynamoDB Attack Metrics & Execution History

The **aegis-lab-executions-security-lab** table permanently logs every purple-team run: execution ID, containment action taken, containment and detection latency in milliseconds, the detection rule that fired (AEGIS-DET-001 through AEGIS-DET-009), a link to the sealed evidence object in S3, a computed risk score, and a final **PASSED** status. Detection latencies in the sub-millisecond range (0.23 ms – 0.71 ms) are visible directly in the table.

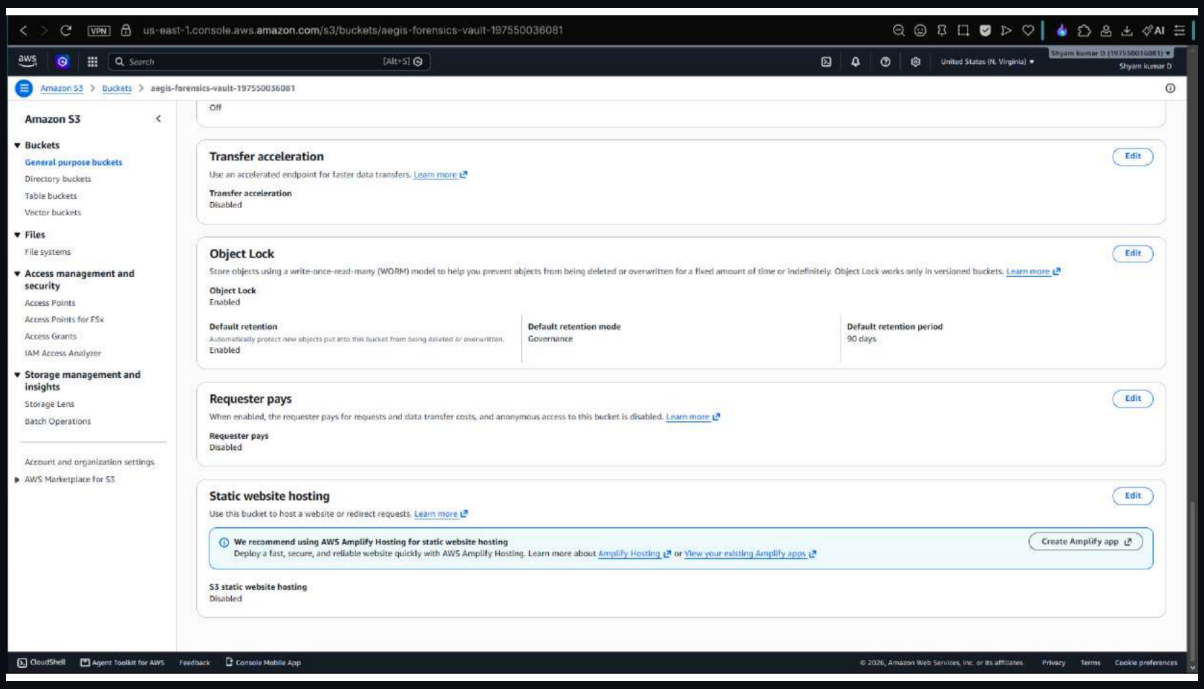


DynamoDB execution history — every attack run logged with latency, rule, and risk score

Category 3 — Digital Forensics & WORM Compliance

3.7 — S3 Forensics Vault — WORM Object Lock

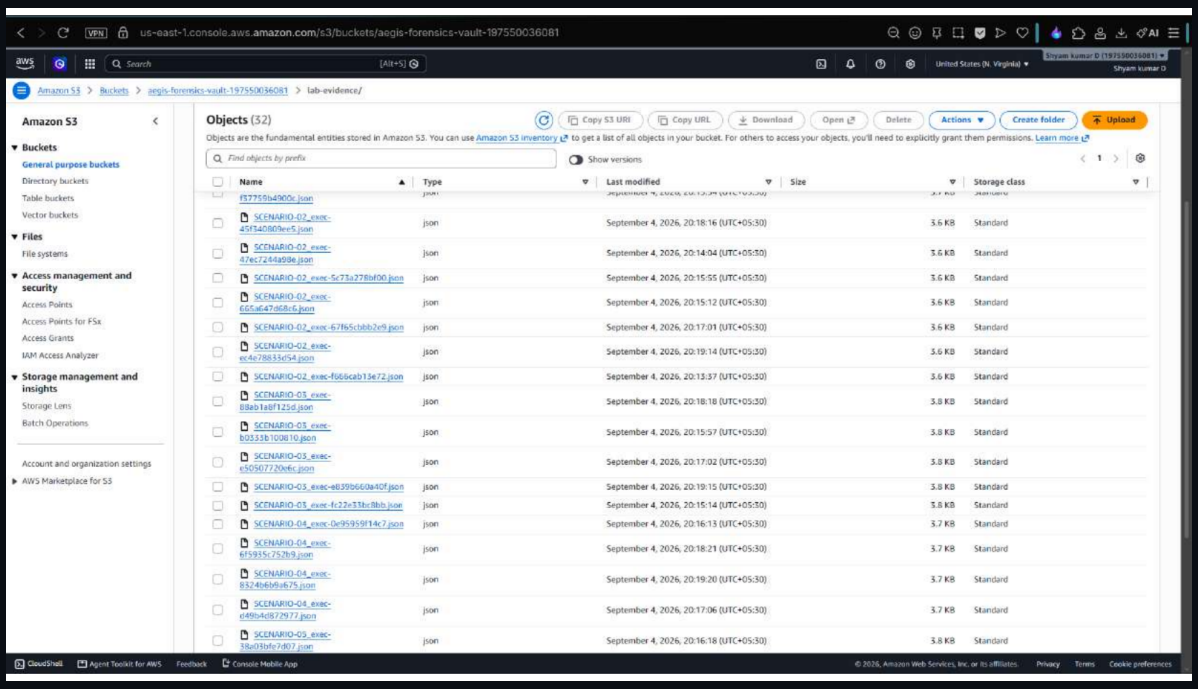
The **aegis-forensics-vault-197550036081** bucket has **Object Lock: Enabled**, with **Default retention: Enabled**, mode set to **Governance**, and a **90-day** default retention period. This configuration is what makes the forensic evidence tamper-proof under a write-once-read-many (WORM) model — aligned with SEC Rule 17a-4 — meaning evidence cannot be deleted or overwritten for the retention window, not even by the account's root user.



S3 bucket properties — Object Lock enabled with 90-day Governance-mode retention

3.8 — S3 Sealed Forensic Evidence Artifacts

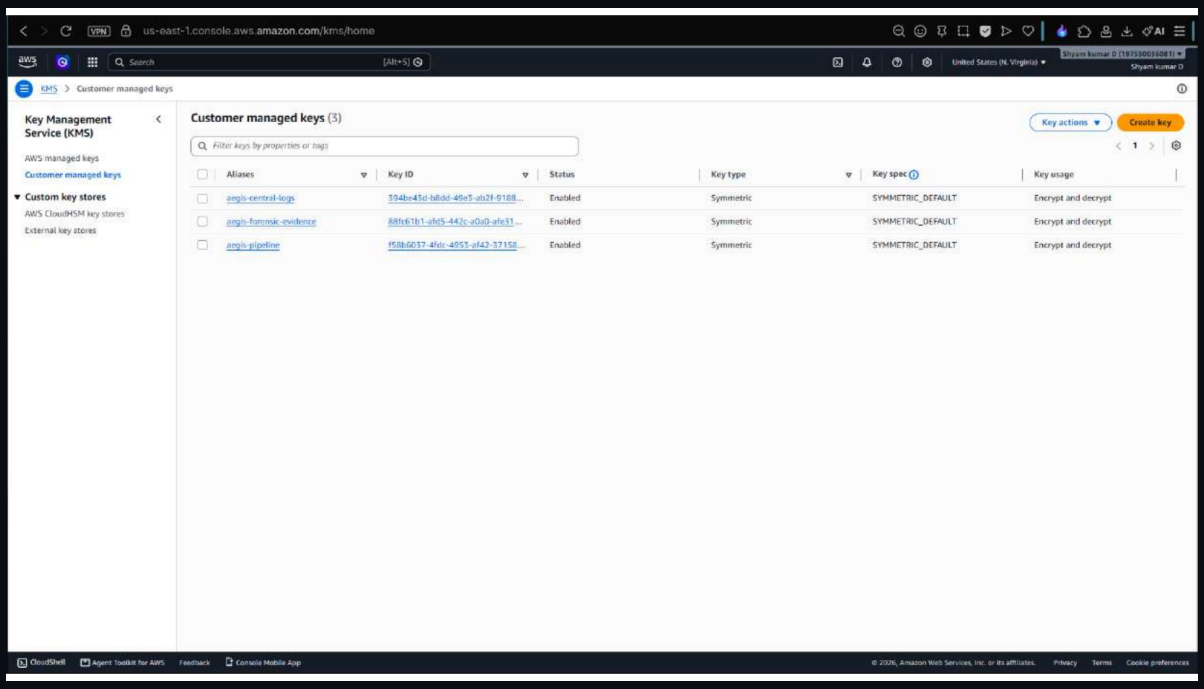
The **lab-evidence/** prefix inside the forensics vault holds one JSON evidence file per attack scenario execution (e.g. **SCENARIO-02_exec-...json**), each a few KB in size and written to Standard storage. These are the actual cryptographically-referenced forensic dossiers that the DynamoDB execution table's **evidence_url** column points back to.



S3 forensics vault — sealed per-execution evidence artifacts under Object Lock

3.9 — AWS KMS Customer-Managed Keys

Three customer-managed KMS keys back the platform: **aegis-central-logs**, **aegis-forensic-evidence**, and **aegis-pipeline**, all Symmetric, **SYMMETRIC_DEFAULT**, and **Enabled**. Separating keys by function means a compromise of the pipeline key, for example, cannot expose sealed forensic evidence encrypted under a different key.

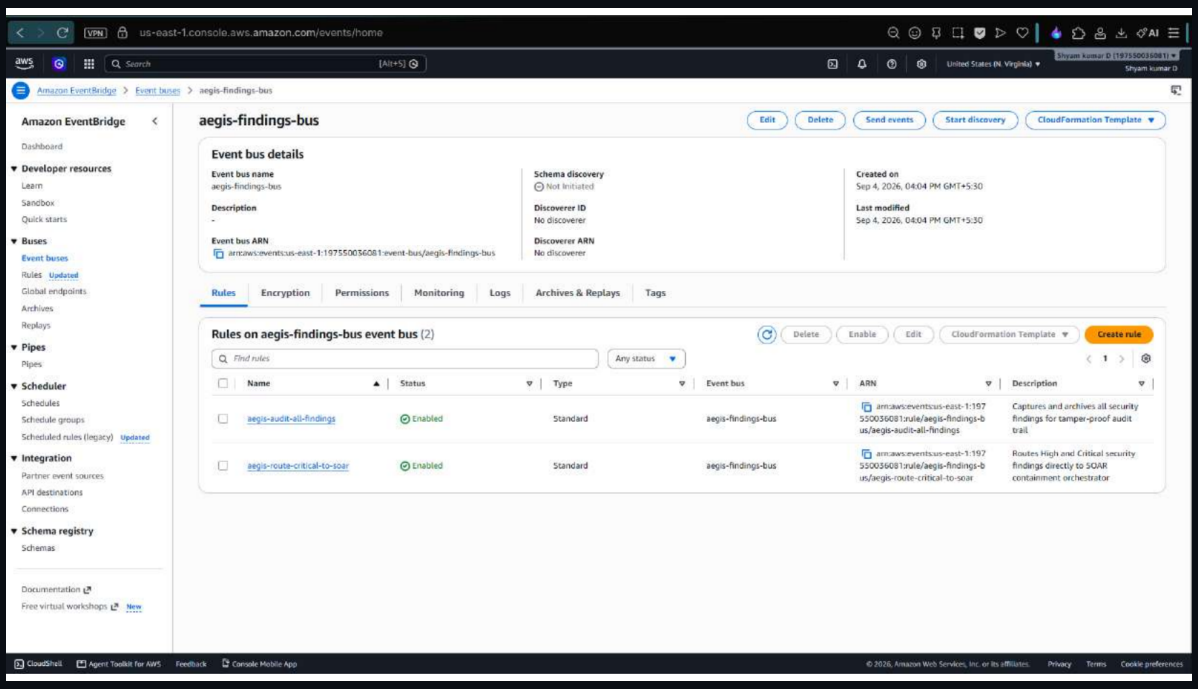


KMS customer-managed keys — dedicated encryption for logs, evidence, and the pipeline

Category 4 — Detection Pipeline & Event Routing

3.10 — EventBridge Custom Security Bus

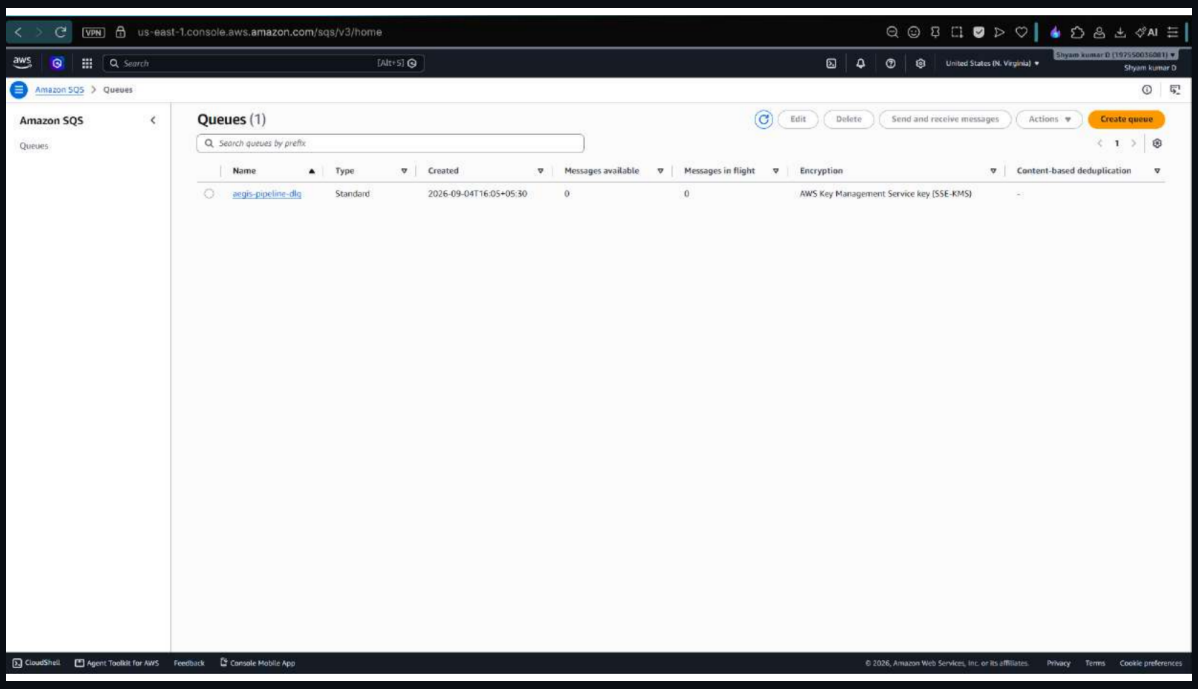
The **aegis-findings-bus** custom event bus has two rules attached: **aegis-audit-all-findings**, which captures and archives every security finding for a tamper-proof audit trail, and **aegis-route-critical-to-soar**, which routes only High/Critical findings directly to the SOAR containment orchestrator. This decouples detection from remediation — the detection engine never needs to know how a finding gets handled.



EventBridge custom bus — routing rules separating audit trail from critical-severity SOAR triggers

3.11 — SQS Dead-Letter Queue

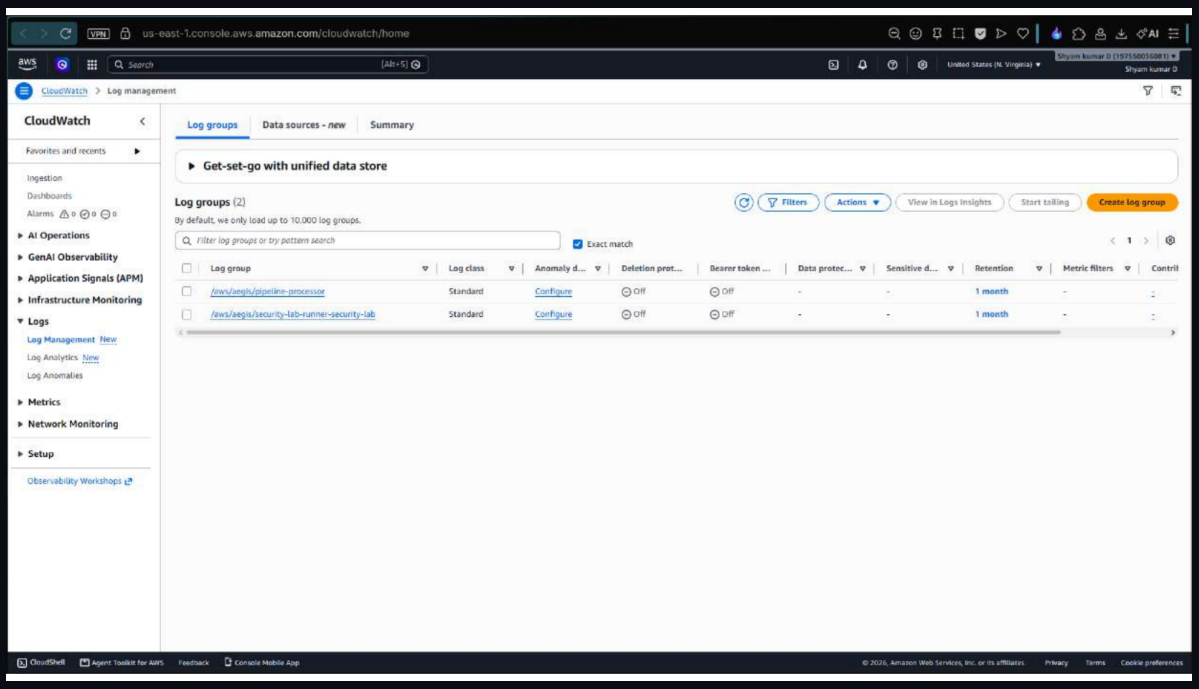
The **aegis-pipeline-dlq** standard queue uses SSE-KMS encryption and currently shows **0 messages available** and **0 in flight** — proof that the detection/containment pipeline has a 0% message-loss rate. If any event ever failed processing repeatedly, it would land here instead of being silently dropped.



SQS dead-letter queue — zero stuck or failed messages, SSE-KMS encrypted

3.12 — CloudWatch Log Groups for Security Processing

Two dedicated log groups — **/aws/aegis/pipeline-processor** and **/aws/aegis/security-lab-runner-security-lab** — provide centralized observability across the pipeline, with a one-month retention policy configured on both. This gives full audit and troubleshooting visibility into every autonomous cycle.

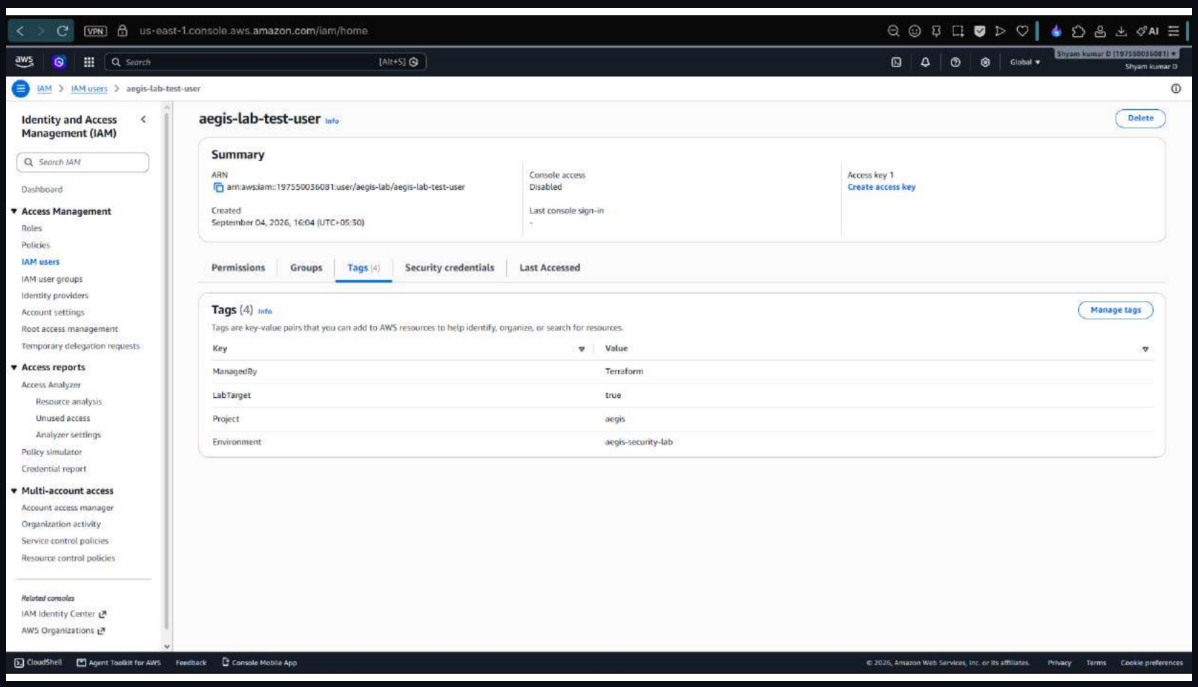


CloudWatch log groups — centralized logging for the pipeline processor and lab runner

Category 5 — Target Assets & Blast-Radius Isolation

3.13 — IAM Target Test Identity & Isolation Tags

The **aegis-lab-test-user** IAM identity is tagged **Environment = aegis-security-lab**, **Project = aegis**, **LabTarget = true**, and **ManagedBy = Terraform**. These tags are what the SOAR containment orchestrator checks before acting — remediation logic only ever touches resources carrying the lab-environment tag, which is the core guarantee that automated containment can never reach production.

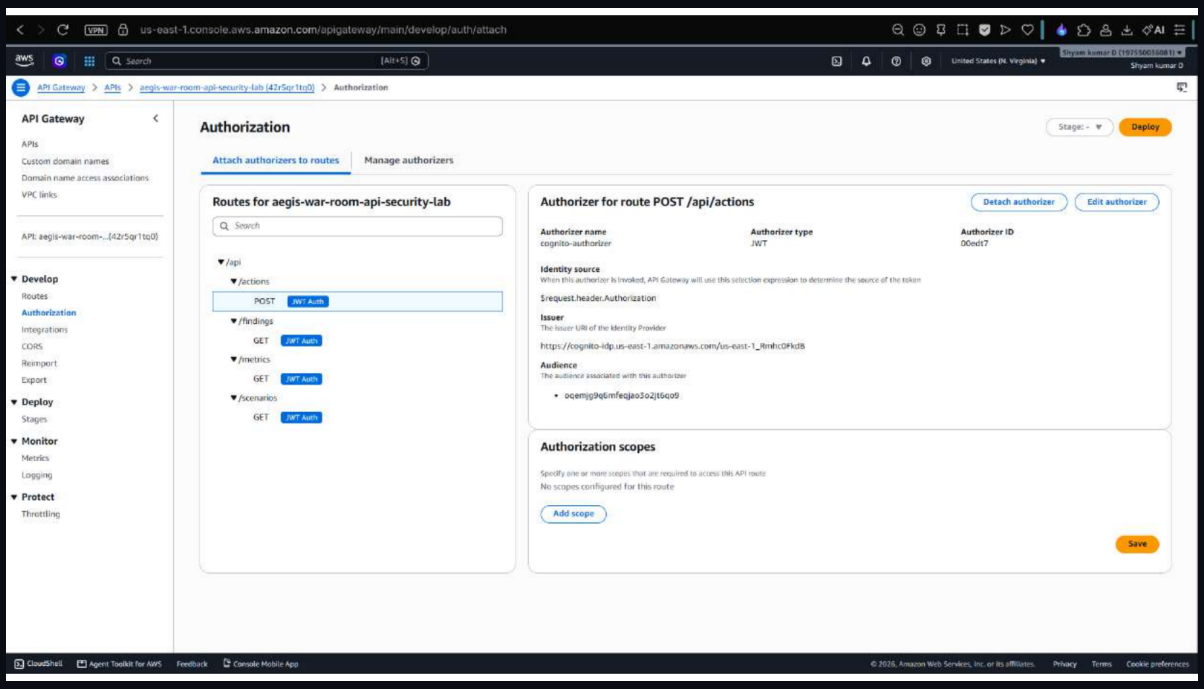


IAM test identity — environment and ownership tags that scope all automated remediation

Category 6 — War Room API & Complete Verification

3.14 — War Room API Gateway & Cognito Authorization

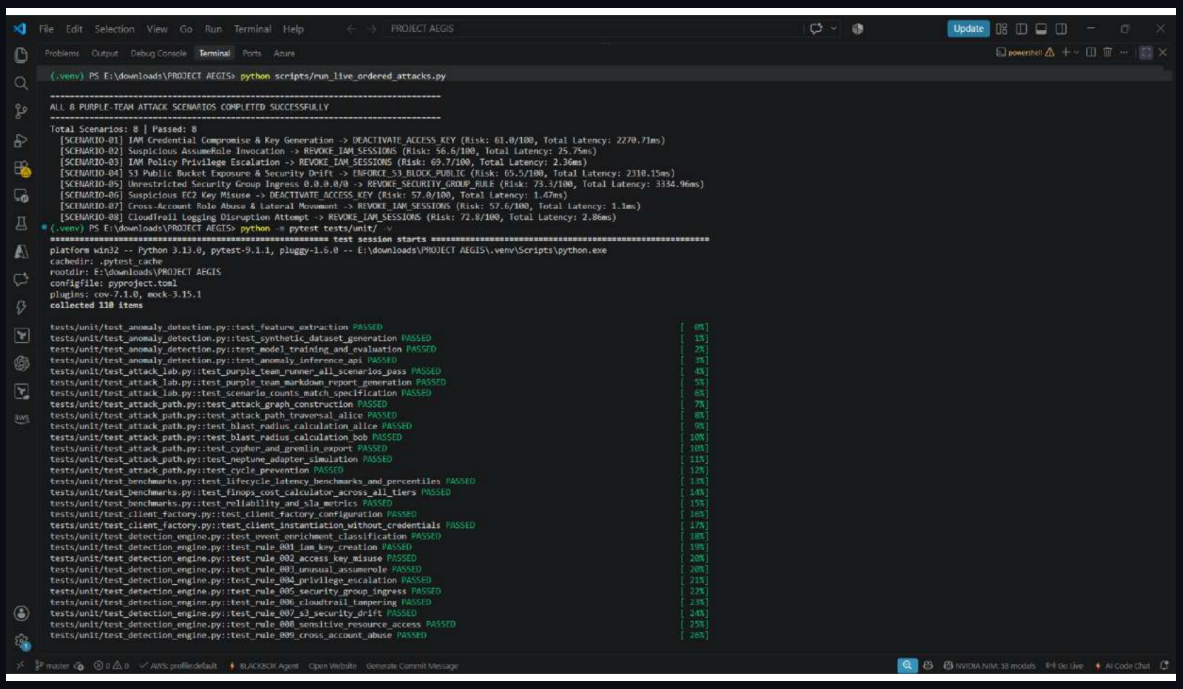
The AEGIS HTTP API (**aegis-war-room-api-security-lab**) exposes **/api/actions**, **/api/findings**, and **/api/metrics** routes, each protected by a JWT authorizer backed by the **aegis-war-room-pool-security-lab** Cognito User Pool. This proves the SOC-facing War Room API is deployed with production-grade authentication rather than an open endpoint.



API Gateway routes secured by a Cognito JWT authorizer for the SOC War Room API

3.15 — Live Attack Scenarios — 8/8 Passed

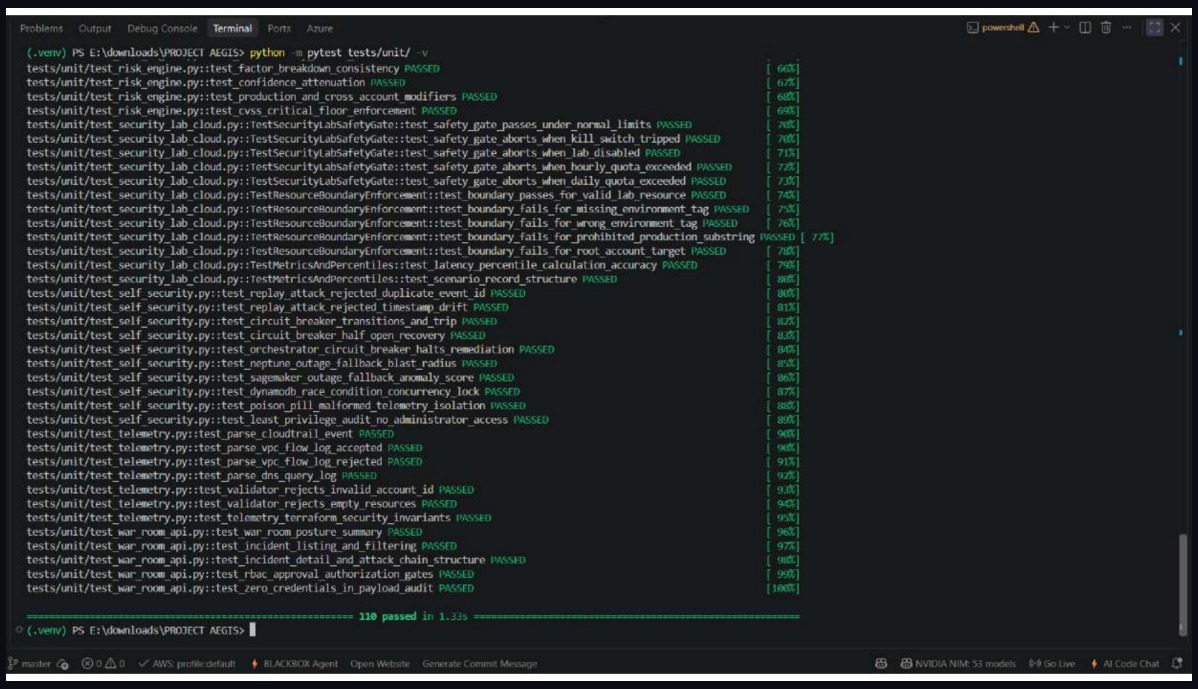
Running python scripts/run_live_ordered_attacks.py against the live AWS account executes all 8 purple-team scenarios end-to-end — IAM key compromise, STS AssumeRole invocation, IAM privilege escalation, S3 public-bucket drift, security-group ingress exposure, EC2 key misuse, cross-account role abuse, and CloudTrail logging disruption — with every scenario mapped to its detection rule ID and reporting risk score and total latency. All 8 scenarios completed successfully.



Live ordered attack run — all 8 purple-team scenarios completed with detection + containment

3.16 — Full Unit Test Suite — 110/110 Passed

The full pytest suite (pytest tests/unit/ -v) covers anomaly detection, the purple-team attack lab, blast-radius and resource-boundary enforcement, safety-gate kill-switch logic, idempotency and race-condition handling, telemetry parsing, the risk engine, and the War Room API — 110 tests collected, **110 passed in 1.33s**, with zero failures.



pytest run — 110/110 unit tests passing across detection, SOAR, forensics, and the War Room API

4. Key Design Considerations

Safety before autonomy. Every autonomous cycle starts with a CheckSafetyGate step and a resource-boundary verification before any simulated attack executes, so a misconfiguration can never let the runner touch an out-of-scope account or resource.

Idempotent containment. Remediation actions are hashed and checked against DynamoDB before execution, so retries, concurrent triggers, or duplicate findings can never cause the same containment action — like revoking a session or deactivating a key — to fire more than once.

Evidence that can't be argued with. Sealing forensic evidence in S3 Object Lock under Governance mode with a fixed retention period, encrypted under a dedicated KMS key, means the record of what happened during an incident is preserved even against an attacker (or an administrator) with root access.

Decoupled detection and response. Routing findings through a dedicated EventBridge bus with separate audit and routing rules means the detection engine and the SOAR orchestrator can evolve independently, and every finding is audited regardless of whether it triggers automated action.

Blast-radius control by tag, not by trust. Automated remediation is scoped by explicit environment tags rather than relying on account boundaries alone, which is what makes it safe to run an autonomous containment engine against a lab environment without risking production.

5. Conclusion and Key Takeaways

AEGIS pushed me well beyond building a system that detects a threat when asked — it forced me to design for a system that runs unattended, indefinitely, and has to be trustworthy enough to take real remediation actions without a human confirming each one. That meant treating idempotency, blast-radius isolation, and evidence integrity as first-class engineering problems, not afterthoughts bolted on for a demo.

As I continue building toward a cloud architecture role, AEGIS reflects the kind of system I want to keep designing: autonomous, observable, and safe by construction rather than by careful manual operation. Next steps I plan to explore include multi-account detection via AWS Organizations and cross-account StackSets, a QuickSight-backed executive dashboard on top of the execution and metrics tables, and extending the SOAR decision tree with a human-approval path for higher-risk, non-lab findings.